

Lab 4

Aron Enström (aroen488), Sergey Volchkov (servo519)

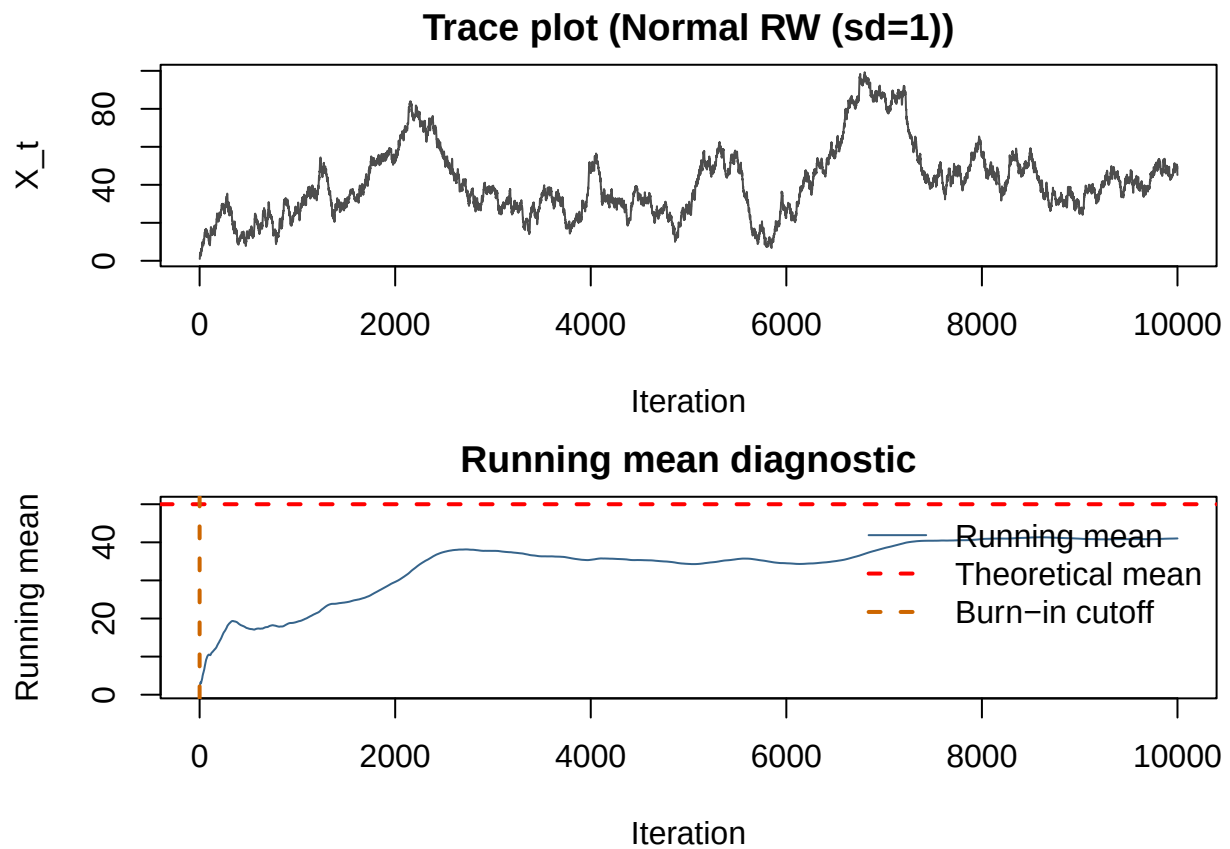
2026-02-11

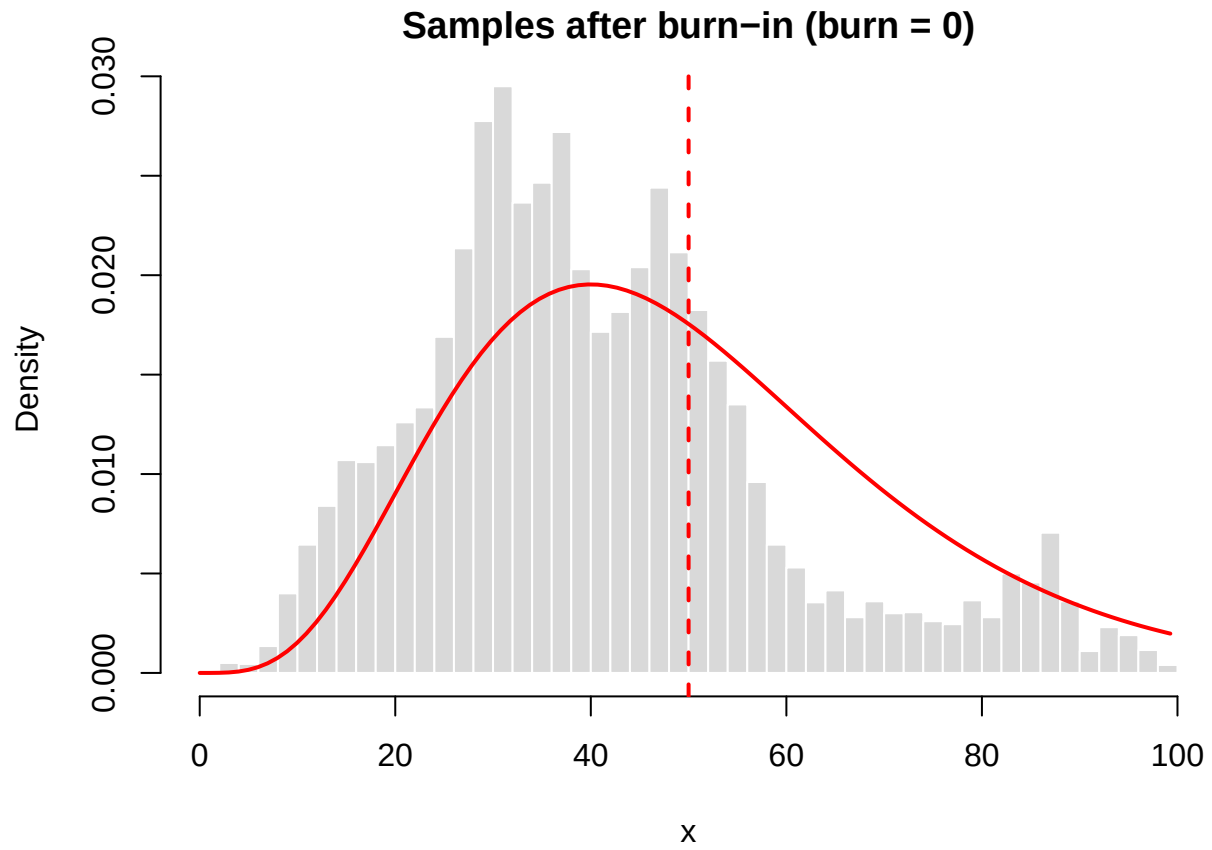
Q1:

a. Metropolis-Hastings with normal distribution as proposal

Proposal: Normal RW (sd=1)

Acceptance rate: 0.9811981

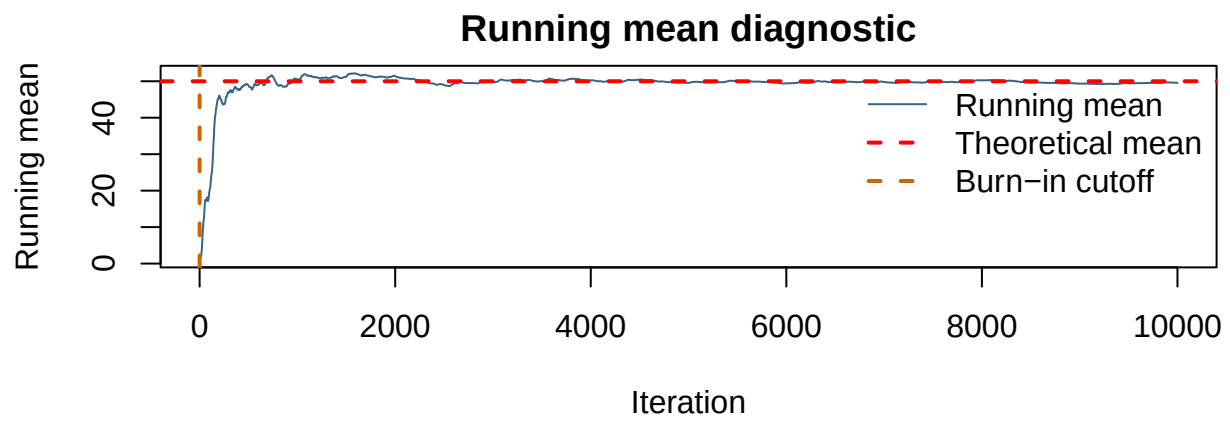
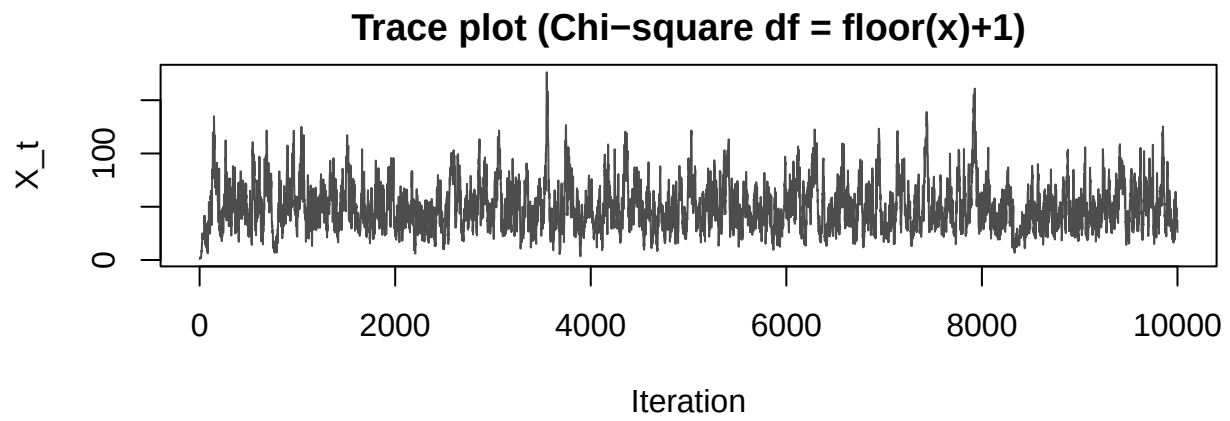


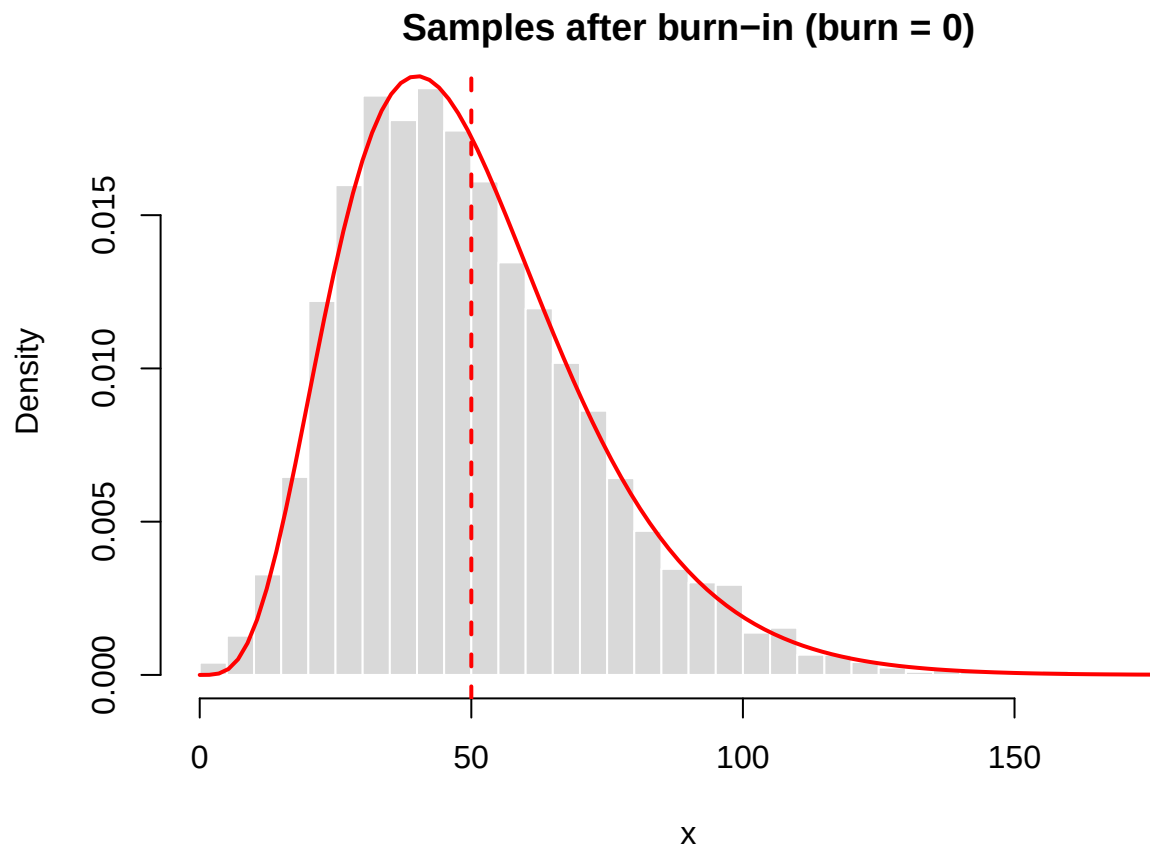


With the standard deviation of 1 and not discarding the samples obtained during the burn-in, we are not seeing convergence to the true mean. The acceptance rate is too high (near 1), which indicates we are moving too slowly.

b. Metropolis-Hastings with chi-square distribution as proposal

```
## Proposal: Chi-square df = floor(x)+1
## Acceptance rate: 0.849985
```

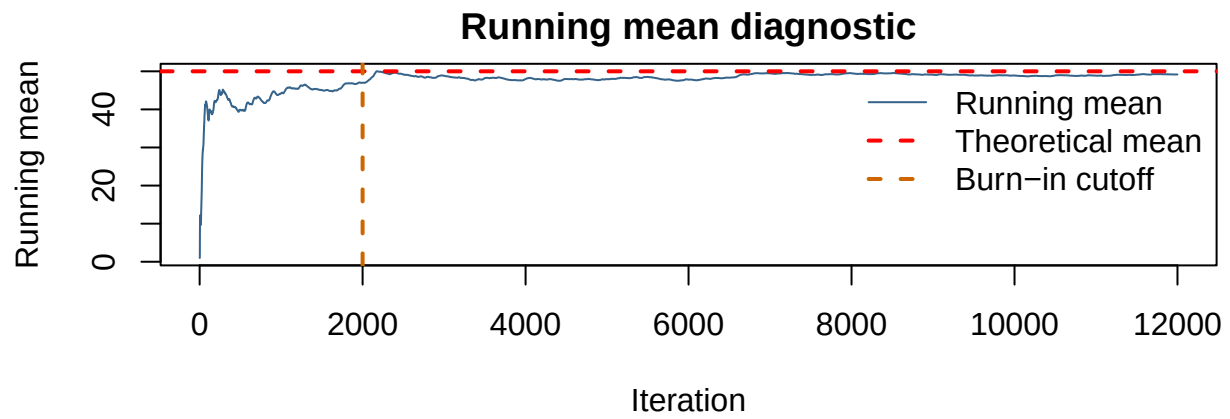
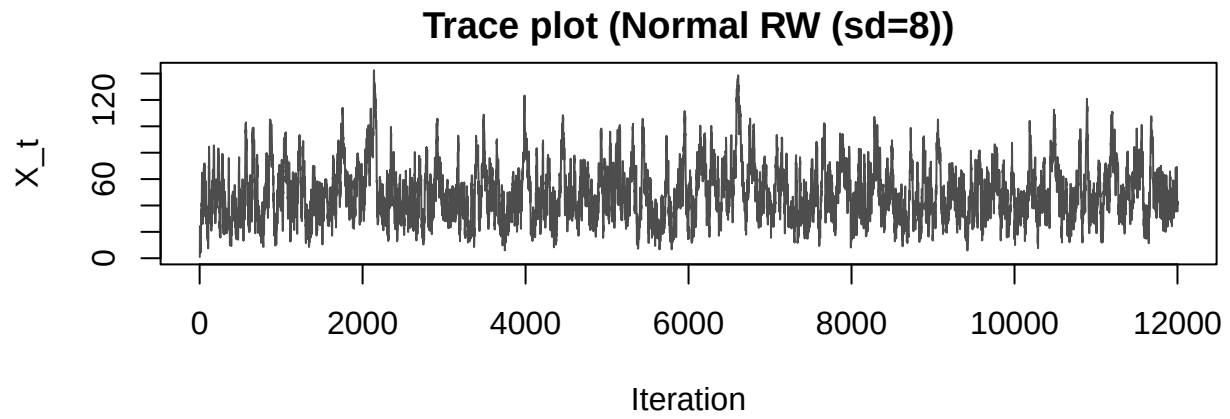


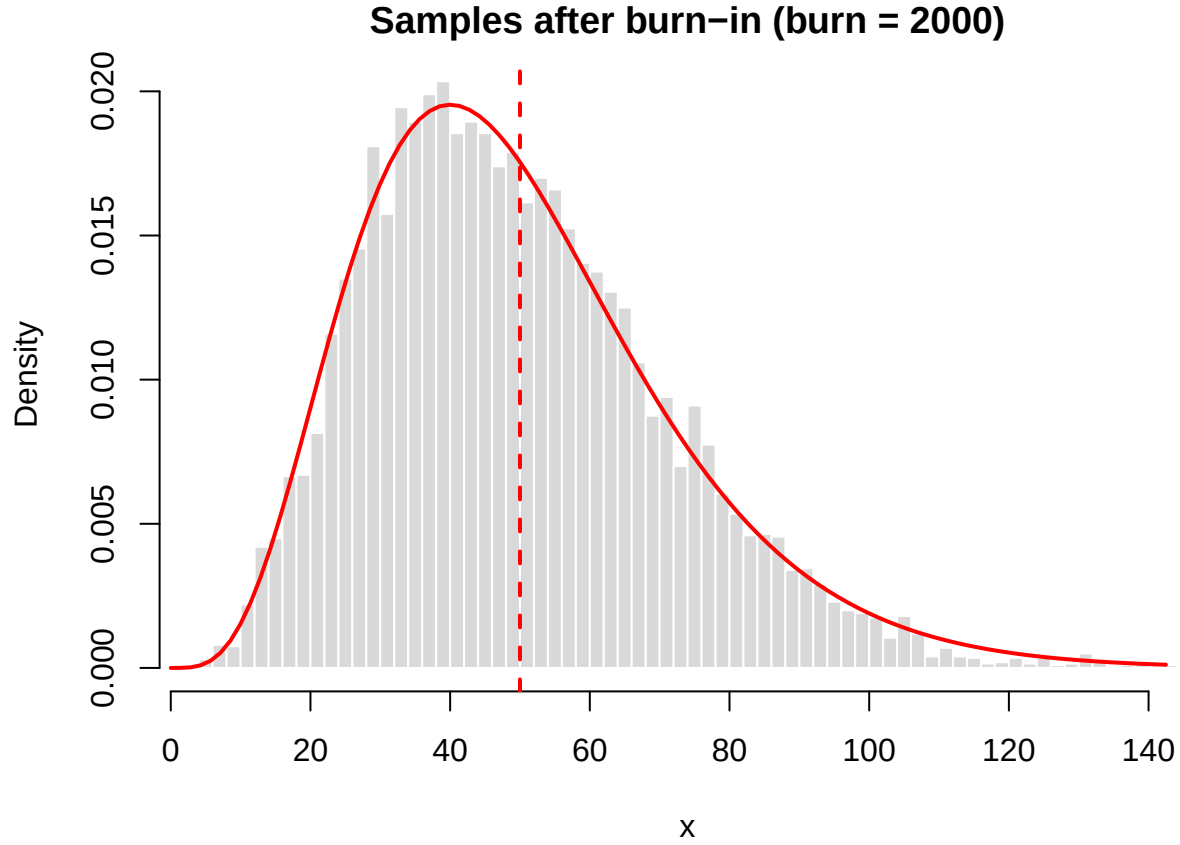


c. Metropolis-Hastings with normal distribution as proposal (sd=8)

Proposal: Normal RW (sd=8)

Acceptance rate: 0.8782399





d. Comparison of the three methods

For $\text{sd} = 8$ and the burn-in period of 2000, we are getting much better picture than with the originally used sd of 1. Our resulting histogram matches the gamma distribution much more closely and the acceptance rate is much lower. However, the chi-squared proposal still seems to be the best in terms of the speed of convergence. The chi-square proposal is state-dependent, e.g. $Y \mid X_t = x \sim \chi^2_{\lfloor x \rfloor + 1}$, so its typical proposal size changes with the current location of the chain. This often yields larger, more effective moves when the chain is in higher-density regions away from 0, so the chain can reach and traverse the typical set in fewer iterations than a fixed-scale random-walk. In contrast, a Normal random-walk proposal $Y \mid X_t = x \sim N(x, \sigma^2)$ has a constant step scale σ , which can lead to very slow exploration when σ is too small (even if acceptance is very high).

e. Comparison of $E(X)$ and medians obtained using the three methods

Table 1: Mean and median of MCMC samples

Chain	Mean	Median
Normal RW (sd=1)	41.021	37.931
Normal RW (sd=8)	49.611	46.872
Chi-square df=floor(x)+1	49.569	46.111

f. Theoretical $E(X)$

For a Gamma distribution parameterized as $X \sim \text{Gamma}(\alpha, \theta)$ (shape α , scale θ), the expectation is $E[X] = \alpha\theta$. Thus for $X \sim \text{Gamma}(5, 10)$, $E[X] = 5 \cdot 10 = 50$ which is the value we used on our plots above.

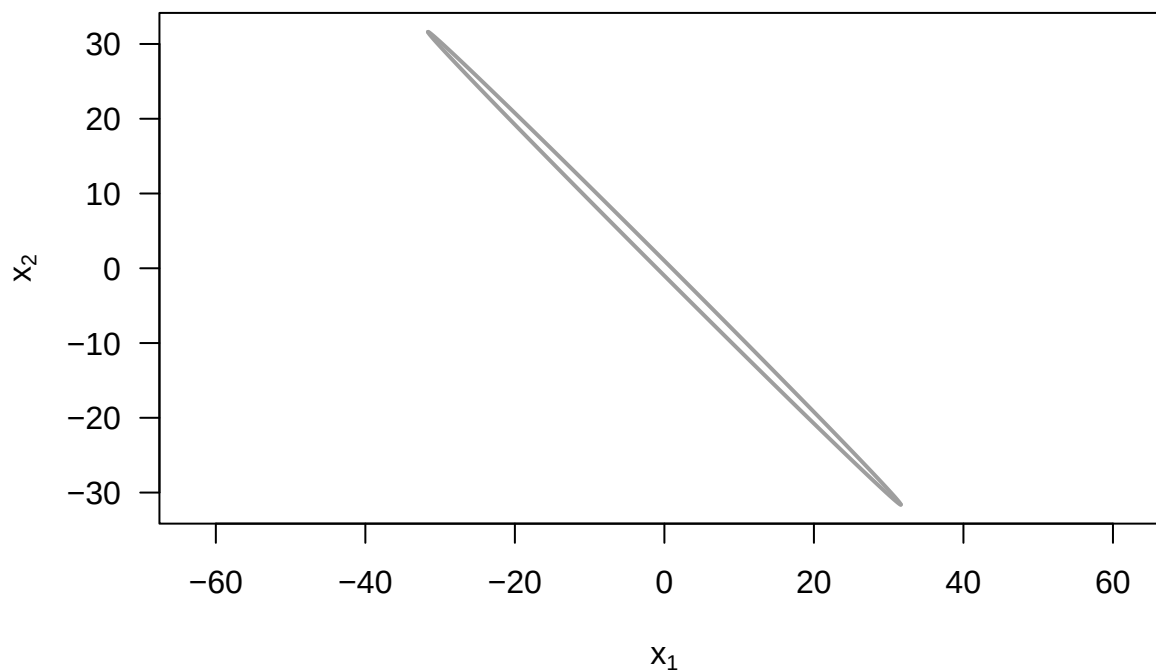
We can see that the chi-square method gets quite close to it (so does normal with $sd = 8$ but only after discarding the first 2000 samples).

We can also double-check the median in R:

```
## [1] 46.70909
```

Q2:

a. Ellipse plot



b. Conditional distributions

We have a bivariate random vector $X = (X_1, X_2)$ that is uniform on the region

$$R_w = \{(x_1, x_2) \in \mathbb{R}^2 : x_1^2 + wx_1x_2 + x_2^2 < 1\}, \quad |w| < 2.$$

If (X_1, X_2) is uniform on a 2D region R , then the conditional distribution of X_1 given $X_2 = x_2$ is uniform on the horizontal slice $T_{x_2} = \{x_1 : (x_1, x_2) \in R\}$ (and similarly with the roles swapped). Equivalently, the conditional density is constant on that slice and equals $1/\text{length}(T_{x_2})$.

Conditional of X_1 given $X_2 = x_2$ Fix x_2 . The constraint $x_1^2 + wx_1x_2 + x_2^2 < 1$ is a quadratic inequality in x_1 :

$$x_1^2 + (wx_2)x_1 + (x_2^2 - 1) < 0.$$

Its discriminant is

$$\Delta_1(x_2) = (wx_2)^2 - 4(x_2^2 - 1) = 4 - (4 - w^2)x_2^2.$$

So the slice is non-empty iff $\Delta_1(x_2) > 0$, i.e.

$$|x_2| < \frac{2}{\sqrt{4-w^2}}.$$

When $\Delta_1(x_2) > 0$, the admissible interval for x_1 is

$$\ell_1(x_2) = \frac{-wx_2 - \sqrt{\Delta_1(x_2)}}{2}, \quad u_1(x_2) = \frac{-wx_2 + \sqrt{\Delta_1(x_2)}}{2}.$$

Therefore,

$$X_1 \mid (X_2 = x_2) \sim \text{Unif}(\ell_1(x_2), u_1(x_2)), \quad \text{for } |x_2| < \frac{2}{\sqrt{4-w^2}}.$$

because a uniform distribution on an interval has constant density on that interval.

The conditional density can be written as

$$f_{X_1|X_2}(x_1 \mid x_2) = \begin{cases} \frac{1}{u_1(x_2) - \ell_1(x_2)} = \frac{1}{\sqrt{\Delta_1(x_2)}}, & \ell_1(x_2) \leq x_1 \leq u_1(x_2), \\ 0, & \text{otherwise.} \end{cases}$$

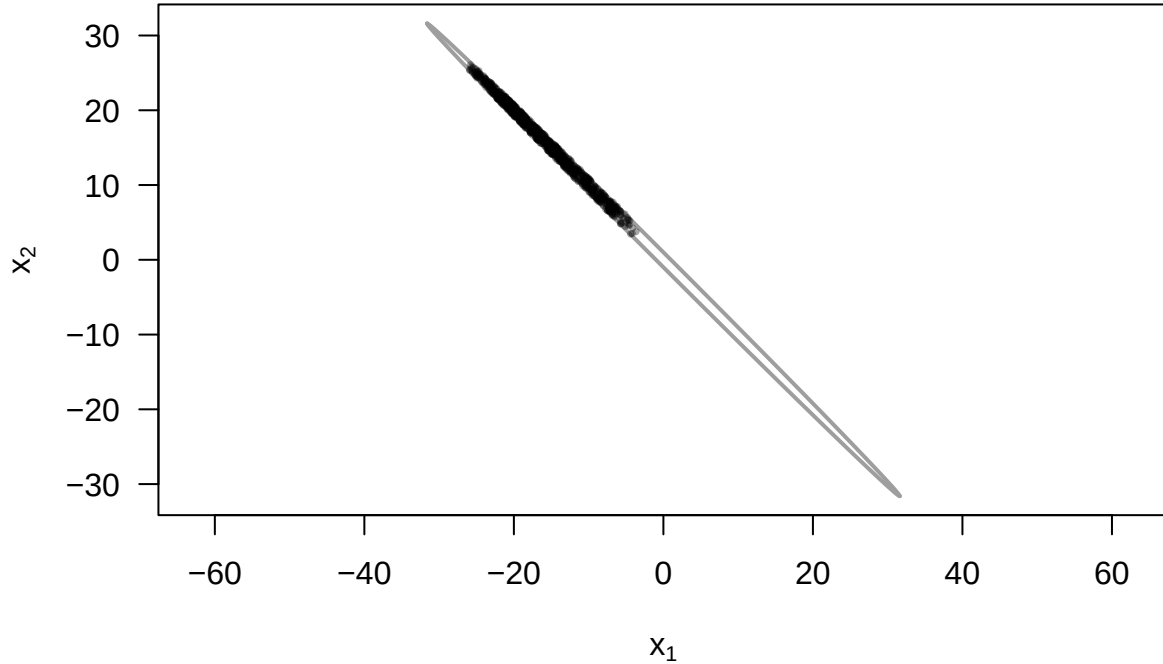
using that a $\text{Unif}(a, b)$ density is $1/(b-a)$ on $[a, b]$.

Similarly,

$$X_2 \mid (X_1 = x_1) \sim \text{Unif}(\ell_2(x_1), u_2(x_1)), \quad \text{for } |x_1| < \frac{2}{\sqrt{4-w^2}}.$$

by the same “uniform on a slice” principle.

c. Gibbs sampling



```
## P(X1 > 0): 0
```

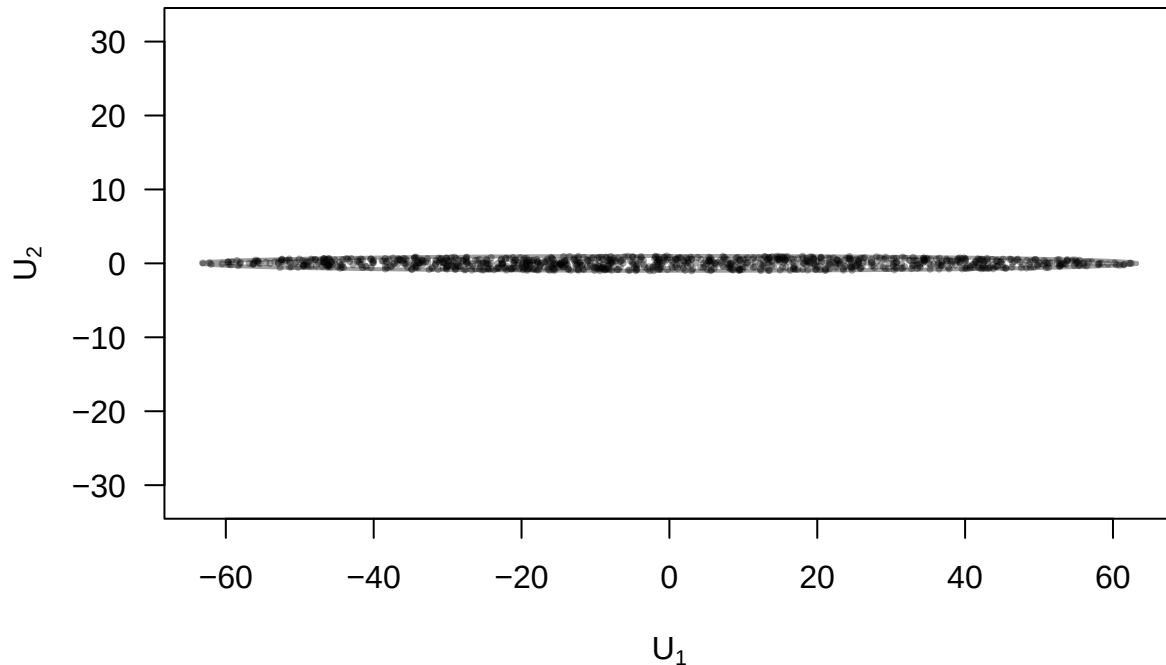
```
## P(X1 > 0) based on repeated sampling: 0.779 0.749 0.039 0.519 0.127
```

By symmetry, the true value of $P(X_1 > 0)$ should be 0.5.

d. Why the results are worse than for $w=1.8$

Gibbs sampling is less successful at $w = 1.999$ because the target region becomes extremely elongated (nearly “degenerate”), so X_1 and X_2 are very strongly coupled and single-coordinate updates move only tiny distances along the long, thin direction—leading to very slow mixing and high autocorrelation.

e. Results for transformed distribution $U = (U_1, U_2) = (X_1 - X_2, X_1 + X_2)$



```
## P(X1 > 0) as P((U2 + U1)/2 > 0): 0.481
```

```
## P(X1 > 0) as P((U2 + U1)/2 > 0) from multiple runs: 0.515 0.517 0.487 0.504 0.499 0.486 0.48 0.516 0
```

We can see that the results are much closer to the true value of $P(X_1 > 0)$ which is equal to 0.5. The estimates are closer to 0.5 because the transformation to $U_1 = X_1 - X_2, U_2 = X_1 + X_2$ rotates the geometry by 45 degrees so the support becomes an axis-aligned ellipse, and then coordinate-wise Gibbs updates are aligned with the ellipse’s principal axes so the chain moves efficiently.

Appendix

```
#####  
# 1(a)  
#####
```

```

## Target:  $f(x) = (1/(2.4 \cdot 10^6)) \cdot x^4 \cdot \exp(-x/10)$ ,  $x > 0$ 
## This is exactly  $\text{Gamma}(\text{shape}=5, \text{scale}=10)$ .
shape_true <- 5
scale_true <- 10

## Use log of *unnormalized* density (constant cancels in MH ratio)
log_f_unnorm <- function(x) {
  ifelse(x > 0, 4 * log(x) - x / 10, -Inf)
}

## Generic MH sampler for arbitrary proposal  $q(y/x)$ 
mh_sampler <- function(n = 10000, x0 = 1, rprop, dprop, ...) {
  x <- numeric(n)
  x[1] <- x0
  acc <- 0L

  for (t in 2:n) {
    xt <- x[t - 1]
    y <- rprop(xt, ...)

    log_num <- log_f_unnorm(y) + log(dprop(xt, y, ...)) #  $f(y) q(x/y)$ 
    log_den <- log_f_unnorm(xt) + log(dprop(y, xt, ...)) #  $f(x) q(y/x)$ 
    log_alpha <- log_num - log_den

    if (log(runif(1)) < log_alpha) {
      x[t] <- y
      acc <- acc + 1L
    } else {
      x[t] <- xt
    }
  }

  list(x = x, acc_rate = acc / (n - 1))
}

## Proposal "factories"
make_norm_rw_proposal <- function(sd = 1) {
  list(
    name = sprintf("Normal RW (sd=%g)", sd),
    rprop = function(x) rnorm(1, mean = x, sd = sd),
    dprop = function(y, x) dnorm(y, mean = x, sd = sd)
  )
}

make_chisq_floor_proposal <- function() {
  list(
    name = "Chi-square df = floor(x)+1",
    rprop = function(x) {
      df <- max(1L, floor(x) + 1L) #  $\text{floor}(x+1) == \text{floor}(x)+1$ 
      rchisq(1, df = df)
    },
    dprop = function(y, x) {
      df <- max(1L, floor(x) + 1L)

```

```

    dchisq(y, df = df)
  }
)
}

run_sampler <- function(proposal, burn = 0, n = 10000, x0 = 1) {
  res <- mh_sampler(n = n, x0 = x0, rprop = proposal$rprop, dprop = proposal$dprop)
  x <- res$x

  cat("Proposal:", proposal$name, "\n")
  cat("Acceptance rate:", res$acc_rate, "\n")

  ## Trace + running mean
  par(mfrow = c(2, 1), mar = c(4, 4, 2, 1))

  plot(x, type = "l", col = "gray30", xlab = "Iteration", ylab = "X_t",
       main = paste("Trace plot (", proposal$name, ")", sep = ""))

  run_mean <- cumsum(x) / seq_along(x)
  theo_mean <- shape_true * scale_true
  yl <- range(c(run_mean, theo_mean), finite = TRUE)

  plot(run_mean, type = "l", col = "steelblue4",
       xlab = "Iteration", ylab = "Running mean",
       main = "Running mean diagnostic", ylim = yl)
  abline(h = theo_mean, col = "red", lwd = 2, lty = 2)
  abline(v = burn, col = "darkorange3", lwd = 2, lty = 2)

  legend("topright",
        legend = c("Running mean", "Theoretical mean", "Burn-in cutoff"),
        col = c("steelblue4", "red", "darkorange3"),
        lty = c(1, 2, 2),
        lwd = c(1, 2, 2),
        bty = "n")

  ## Post burn-in sample
  x_post <- if (burn > 0) x[-seq_len(burn)] else x

  ## Histogram + true density overlay
  par(mfrow = c(1, 1), mar = c(4, 4, 2, 1))
  hist(x_post, breaks = 50, freq = FALSE, col = "gray85", border = "white",
       main = sprintf("Samples after burn-in (burn = %d)", burn),
       xlab = "x")

  curve(dgamma(x, shape = shape_true, scale = scale_true),
        from = 0, to = max(x_post),
        add = TRUE, col = "red", lwd = 2)

  abline(v = theo_mean, col = "red", lwd = 2, lty = 2)

  x_post <- if (burn > 0) x[-seq_len(burn)] else x

  invisible(list(

```

```

    x = x,
    x_post = x_post,
    acc_rate = res$acc_rate,
    burn = burn,
    proposal = proposal$name
  ))
}
set.seed(1)
prop_norm <- make_norm_rw_proposal(sd = 1)
out_norm_sd1 <- run_sampler(prop_norm, burn = 0, n = 10000, x0 = 1)
x_norm_sd1 <- out_norm_sd1$x_post
#####
# 1(b)
#####
set.seed(1)
prop_chi <- make_chisq_floor_proposal()
out_chi <- run_sampler(prop_chi, burn = 0, n = 10000, x0 = 1)
x_chi <- out_chi$x_post
#####
# 1(c)
#####
set.seed(1)
prop_norm <- make_norm_rw_proposal(sd = 8)
out_norm_sd8 <- run_sampler(prop_norm, burn = 2000, n = 12000, x0 = 1)
x_norm_sd8 <- out_norm_sd8$x_post
#####
# 1(e)
#####
summ_tab <- data.frame(
  Chain = c("Normal RW (sd=1)", "Normal RW (sd=8)", "Chi-square df=floor(x)+1"),
  Mean = c(mean(x_norm_sd1, na.rm = TRUE),
            mean(x_norm_sd8, na.rm = TRUE),
            mean(x_chi, na.rm = TRUE)),
  Median = c(median(x_norm_sd1, na.rm = TRUE),
             median(x_norm_sd8, na.rm = TRUE),
             median(x_chi, na.rm = TRUE))
)

knitr::kable(summ_tab, digits = 3, caption = "Mean and median of MCMC samples")
#####
# 1.6
#####
qgamma(0.5, shape=5, scale=10)
#####
# 2(a)
#####
w <- 1.999

a <- 1 - w^2/4
xmax <- 1/sqrt(a)
xv <- seq(-xmax, xmax, length.out = 4000)

disc <- pmax(0, 1 - a * xv^2) # guard against tiny negatives from rounding
ylo <- -(w/2) * xv - sqrt(disc)

```

```

yhi <- -(w/2) * xv + sqrt(disc)

plot(xv, ylo, type = "n",
     xlab = expression(x[1]), ylab = expression(x[2]), las = 1,
     asp = 1,
     xlim = range(xv), ylim = range(c(ylo, yhi)))

lines(xv, ylo, lwd = 2, col = 8)
lines(xv, yhi, lwd = 2, col = 8)

#####
# 2(c)
#####
w <- 1.999
tol <- 1e-12

Q <- function(x1, x2, w) x1^2 + w * x1 * x2 + x2^2

bounds_x1_given_x2 <- function(x2, w, tol = 1e-12) {
  Delta <- 4 - (4 - w^2) * x2^2
  if (Delta < -tol) stop("Infeasible x2 (outside support): discriminant < 0.")
  Delta <- max(0, Delta)
  s <- sqrt(Delta)
  lo <- (-w * x2 - s) / 2
  hi <- (-w * x2 + s) / 2
  c(lower = min(lo, hi), upper = max(lo, hi))
}

bounds_x2_given_x1 <- function(x1, w, tol = 1e-12) {
  Delta <- 4 - (4 - w^2) * x1^2
  if (Delta < -tol) stop("Infeasible x1 (outside support): discriminant < 0.")
  Delta <- max(0, Delta)
  s <- sqrt(Delta)
  lo <- (-w * x1 - s) / 2
  hi <- (-w * x1 + s) / 2
  c(lower = min(lo, hi), upper = max(lo, hi))
}

gibbs_Rw <- function(n = 1000, burn = 2000, w = 1.999, x0 = c(0, 0), tol = 1e-12) {
  x1 <- x0[1]; x2 <- x0[2]
  if (Q(x1, x2, w) >= 1) stop("Initial state x0 must satisfy Q(x0) < 1.")

  out <- matrix(NA_real_, nrow = n, ncol = 2)
  colnames(out) <- c("X1", "X2")

  for (i in seq_len(burn + n)) {
    b1 <- bounds_x1_given_x2(x2, w, tol)
    x1 <- runif(1, min = b1["lower"], max = b1["upper"])

    b2 <- bounds_x2_given_x1(x1, w, tol)
    x2 <- runif(1, min = b2["lower"], max = b2["upper"])

    # Safety: if rounding nudges us outside, scale back inside (Q(c x)=c^2 Q(x))

```

```

q <- Q(x1, x2, w)
if (!is.finite(q)) stop("Non-finite Q encountered.")
if (q >= 1) {
  cscale <- sqrt((1 - 10 * tol) / q)
  x1 <- x1 * cscale
  x2 <- x2 * cscale
}

if (i > burn) out[i - burn, ] <- c(x1, x2)
}
out
}

set.seed(1)
samp <- gibbs_Rw(n = 1000, burn = 5000, w = w)

a <- 1 - w^2 / 4
xmax <- 1 / sqrt(a)
xv <- seq(-xmax, xmax, length.out = 4000)
disc <- pmax(0, 1 - a * xv^2)
ylo <- -(w / 2) * xv - sqrt(disc)
yhi <- -(w / 2) * xv + sqrt(disc)

plot(xv, ylo, type = "n",
      xlab = expression(x[1]), ylab = expression(x[2]),
      asp = 1, las = 1,
      xlim = range(xv), ylim = range(c(ylo, yhi)))
lines(xv, ylo, lwd = 2, col = 8)
lines(xv, yhi, lwd = 2, col = 8)
points(samp[, "X1"], samp[, "X2"], pch = 16, cex = 0.5, col = rgb(0, 0, 0, 0.35))

# Estimate P(X1 > 0)
p_hat <- mean(samp[, "X1"] > 0)
cat("P(X1 > 0):", p_hat)

set.seed(2)
reps <- 5
p_hats <- replicate(reps, {
  s <- gibbs_Rw(n = 1000, burn = 5000, w = w)
  mean(s[, "X1"] > 0)
})
cat("P(X1 > 0) based on repeated sampling:", p_hats)

#####
# 2(e)
#####
w <- 1.999
n <- 1000
burn <- 5000
tol <- 1e-12

Qx <- function(x1, x2, w) x1^2 + w * x1 * x2 + x2^2
Qu <- function(u1, u2, w) (2 - w) * u1^2 + (2 + w) * u2^2

```

```

X_from_U <- function(u1, u2) c(x1 = (u2 + u1) / 2, x2 = (u2 - u1) / 2)
U_from_X <- function(x1, x2) c(u1 = x1 - x2, u2 = x1 + x2)

a1 <- sqrt(4 / (2 - w)) # max |u1|
a2 <- sqrt(4 / (2 + w)) # max |u2|

tt <- seq(0, 2 * pi, length.out = 4000)
u1_boundary <- a1 * cos(tt)
u2_boundary <- a2 * sin(tt)

plot(u1_boundary, u2_boundary, type = "l", lwd = 2, col = 8,
     xlab = expression(U[1]), ylab = expression(U[2]),
     asp = 1, las = 1)

bounds_u1_given_u2 <- function(u2, w, tol = 1e-12) {
  denom <- (2 - w)
  num <- 4 - (2 + w) * u2^2
  if (num < -tol) stop("Infeasible u2 (outside support).")
  num <- max(0, num)
  r <- sqrt(num / denom)
  c(lower = -r, upper = r)
}

bounds_u2_given_u1 <- function(u1, w, tol = 1e-12) {
  denom <- (2 + w)
  num <- 4 - (2 - w) * u1^2
  if (num < -tol) stop("Infeasible u1 (outside support).")
  num <- max(0, num)
  r <- sqrt(num / denom)
  c(lower = -r, upper = r)
}

gibbs_U <- function(n = 1000, burn = 5000, w = 1.999, u0 = c(0, 0), tol = 1e-12) {
  u1 <- u0[1]; u2 <- u0[2]
  if (Qu(u1, u2, w) >= 4) stop("Initial u0 must satisfy (2-w)u1^2 + (2+w)u2^2 < 4.")

  out <- matrix(NA_real_, nrow = n, ncol = 2)
  colnames(out) <- c("U1", "U2")

  for (i in seq_len(burn + n)) {
    b1 <- bounds_u1_given_u2(u2, w, tol)
    u1 <- runif(1, min = b1["lower"], max = b1["upper"])

    b2 <- bounds_u2_given_u1(u1, w, tol)
    u2 <- runif(1, min = b2["lower"], max = b2["upper"])

    # Numerical safety: nudge strictly inside if needed (scale (u1,u2) since Qu is quadratic)
    q <- Qu(u1, u2, w)
    if (!is.finite(q)) stop("Non-finite quadratic form encountered.")
    if (q >= 4) {
      cscale <- sqrt((4 - 10 * tol) / q)
      u1 <- u1 * cscale
      u2 <- u2 * cscale
    }
  }
}

```

```

    }

    if (i > burn) out[i - burn, ] <- c(u1, u2)
  }
  out
}

set.seed(1)
Usamp <- gibbs_U(n = n, burn = burn, w = w)

points(Usamp[, "U1"], Usamp[, "U2"], pch = 16, cex = 0.5, col = rgb(0, 0, 0, 0.35))

## Estimate  $P(X_1 > 0)$  using  $U$ 
#  $X_1 = (U_2 + U_1)/2$ , so event  $X_1 > 0$  is equivalent to  $U_1 + U_2 > 0$ .
p_hat_U <- mean((Usamp[, "U1"] + Usamp[, "U2"]) > 0)
cat("P( $X_1 > 0$ ) as P( $(U_2 + U_1)/2 > 0$ ):", p_hat_U)

set.seed(123)

reps <- 10
p_hats_U <- replicate(reps, {
  U <- gibbs_U(n = 1000, burn = 5000, w = w)
  mean((U[, "U1"] + U[, "U2"]) > 0)
})

cat("P( $X_1 > 0$ ) as P( $(U_2 + U_1)/2 > 0$ ) from multiple runs:", p_hats_U)

```